

CosmicWatch Edge Hardware Benchmarks

On-device results: Raspberry Pi 4 vs Jetson Orin Nano — meeting brief, 2026-07-30

Headline results

1. The 49-parameter event classifier runs ~4M events/s on a Pi 4 – about 3,000,000x the real detector rate (1.38 Hz). Inference compute is effectively free on any of these boards.
2. Event-driven gateway on Pi 4: CPU duty fraction at the real event rate is 0.04% – board idle power, not compute, is the entire deployment power budget.
3. FL client (26,584-param autoencoder): Jetson CPU is ~1.8x faster per steady-state round than its own GPU (launch/transfer overhead dominates at this model size), but the GPU trains at barely above idle power and wins ~2.2x on active energy (0.08 vs 0.17 J/round).
4. Jetson idle floor is 3.18 W and software config cannot reduce it (headless vs desktop is a null result; 7W mode saves only ~0.16 W). Low-power deployment needs different hardware, e.g. a microcontroller-class device for the tiny classifier.
5. A Jetson FL client is only justified by GPU-sized models (images / larger GNN); for this workload the Pi 4 at lower idle power remains the right always-on gateway.

Table 1 — FL client local-training benchmark
(real legacy device 13: 2,400 train images, batch 256, 1 local epoch/round)

Board / device	6-round s/round	steady s/round	load W	active W	steady J/round (active)	peak RSS MiB
Pi 4 (4x A72, CPU)	0.900	0.161	not meas.*	—	—	323
Jetson 15W (6x A78AE, CPU)	0.438	0.063	5.95	2.77	0.17	757
Jetson 15W (Orin iGPU, CUDA)	0.481	0.110	3.89	0.71	0.08	1097

* Pi 4 has no software-readable power sensor (vcgencmd reports no current); an inline USB-C meter is required — the result JSON honestly leaves measurement_required set.

Steady-state excludes the one-time ~4 s torch lazy-init cost paid in round 1 (Fig. 3).
Power: INA3221 VDD_IN median, 5 Hz sysfs sampling, taken mid-run during 1000-round (CPU) / 700-round (CUDA) sustained probes.

Table 2 — Tiny event-classifier inference (MLP 4→8→1, 49 params, 49 bytes int8)

Board	numpy us/event	numpy events/s	pure-Python us/event	headroom vs 1.38 Hz detector	active energy per inference
Raspberry Pi 4	75.9	4,218,034	29.9	3,066,100x	not measured*
Raspberry Pi 400	78.2	3,778,277	29.3	2,746,440x	not measured*
Jetson Orin Nano 7W	60.7	5,528,289	24.3	4,018,528x	0.052 uJ
Jetson Orin Nano 15W	38.4	8,815,617	15.3	6,408,096x	0.074 uJ

Pi 400 result carries throttled=0x50000 (past undervoltage) — treat as indicative only.

Federated-learning client benchmark — Pi 4 vs Jetson Orin Nano (15W)

Fig. 1 — Steady-state round time
Jetson CPU ~1.8x faster than its own GPU

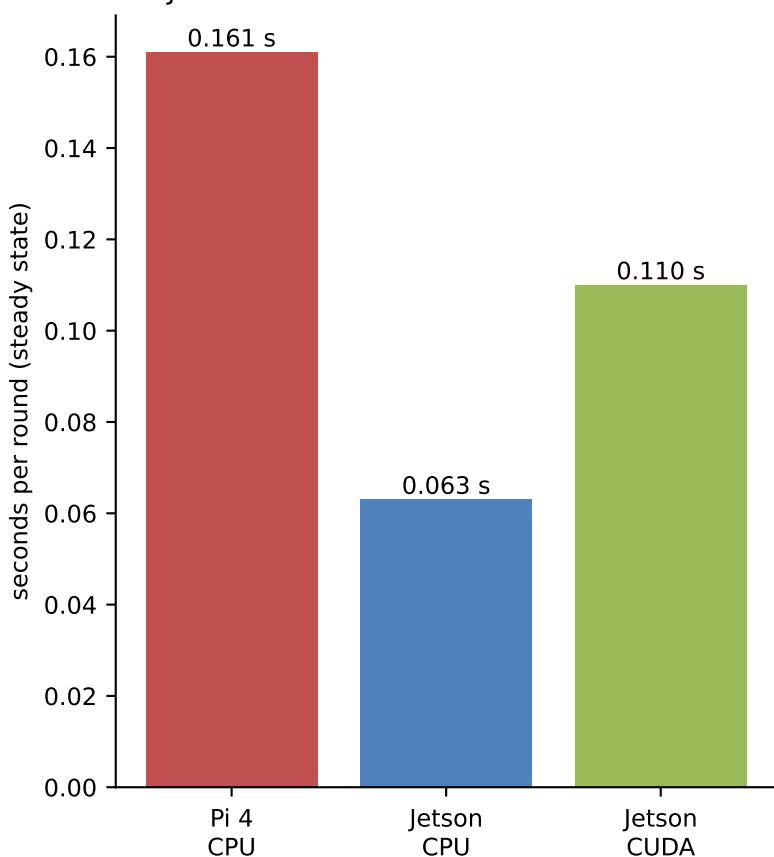


Fig. 2 — Energy per FL round (active)
GPU wins ~2.2x; both are < 0.5 J

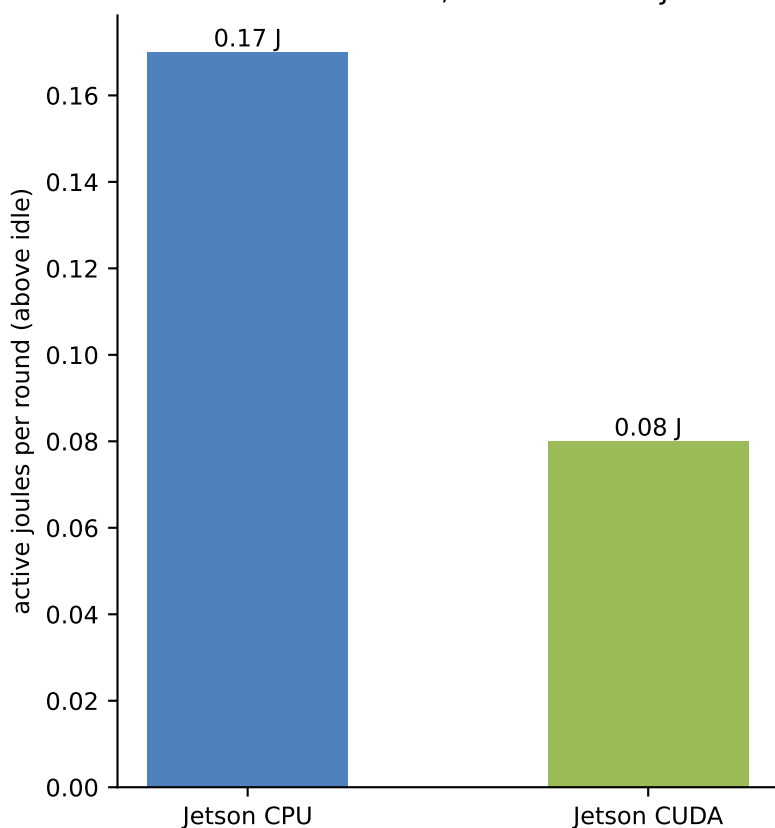


Fig. 3 — Per-round time, 6-round run
round 1 carries ~2-4.6 s one-time torch warmup

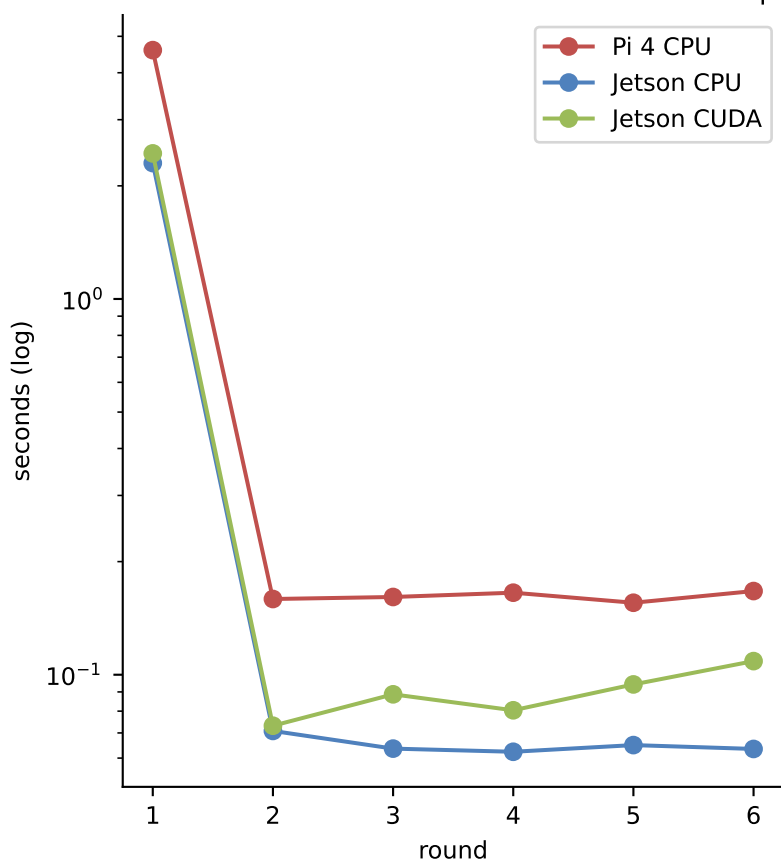
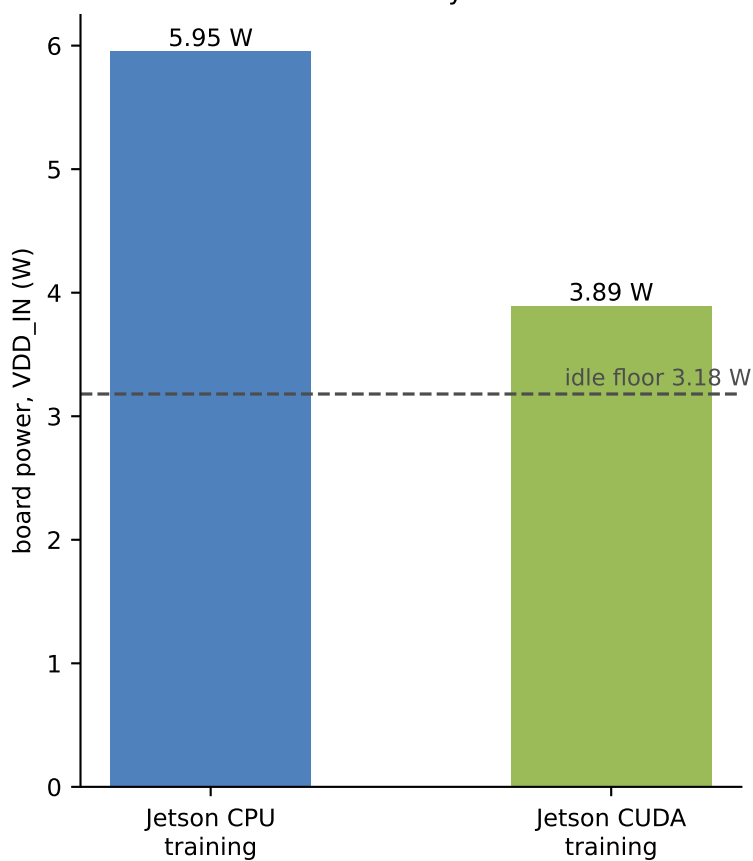


Fig. 4 — Board power under training load
CUDA trains barely above idle



What the FL client trains on — legacy CREDO 20×20 hit-crop images

The benchmarked workload is image data: 65,994 deduplicated CREDO hit-crops split into 8 FL clients by real device ID. The timed client (device 13) trains a 26,584-param autoencoder (400→32→8→32→400, weighted MSE emphasizing bright pixels) on its 2,400 images.

Fig. 9 — 30 random training images from the benchmarked client (device 13, n=2,400)

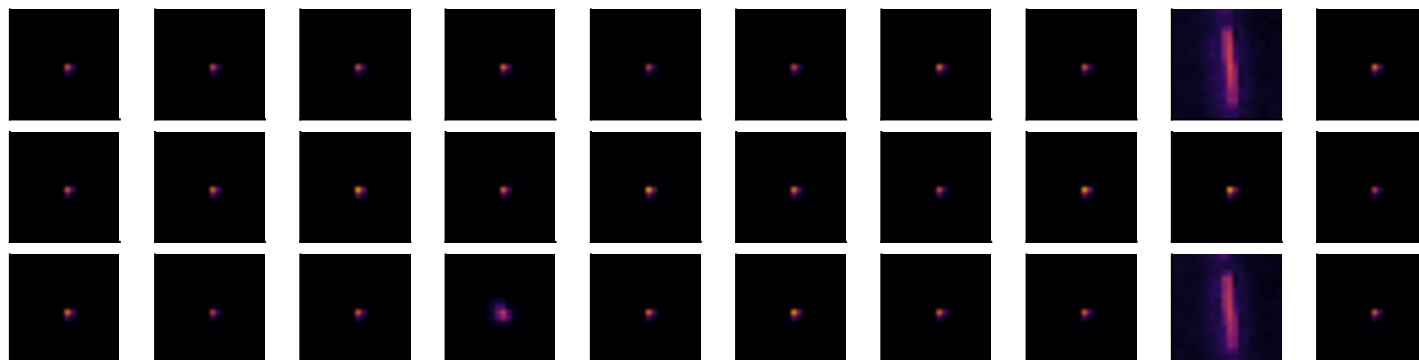


Fig. 10 — The federation is non-IID: 8 real devices, unequal sizes, distinct morphologies (right column: per-client mean image)

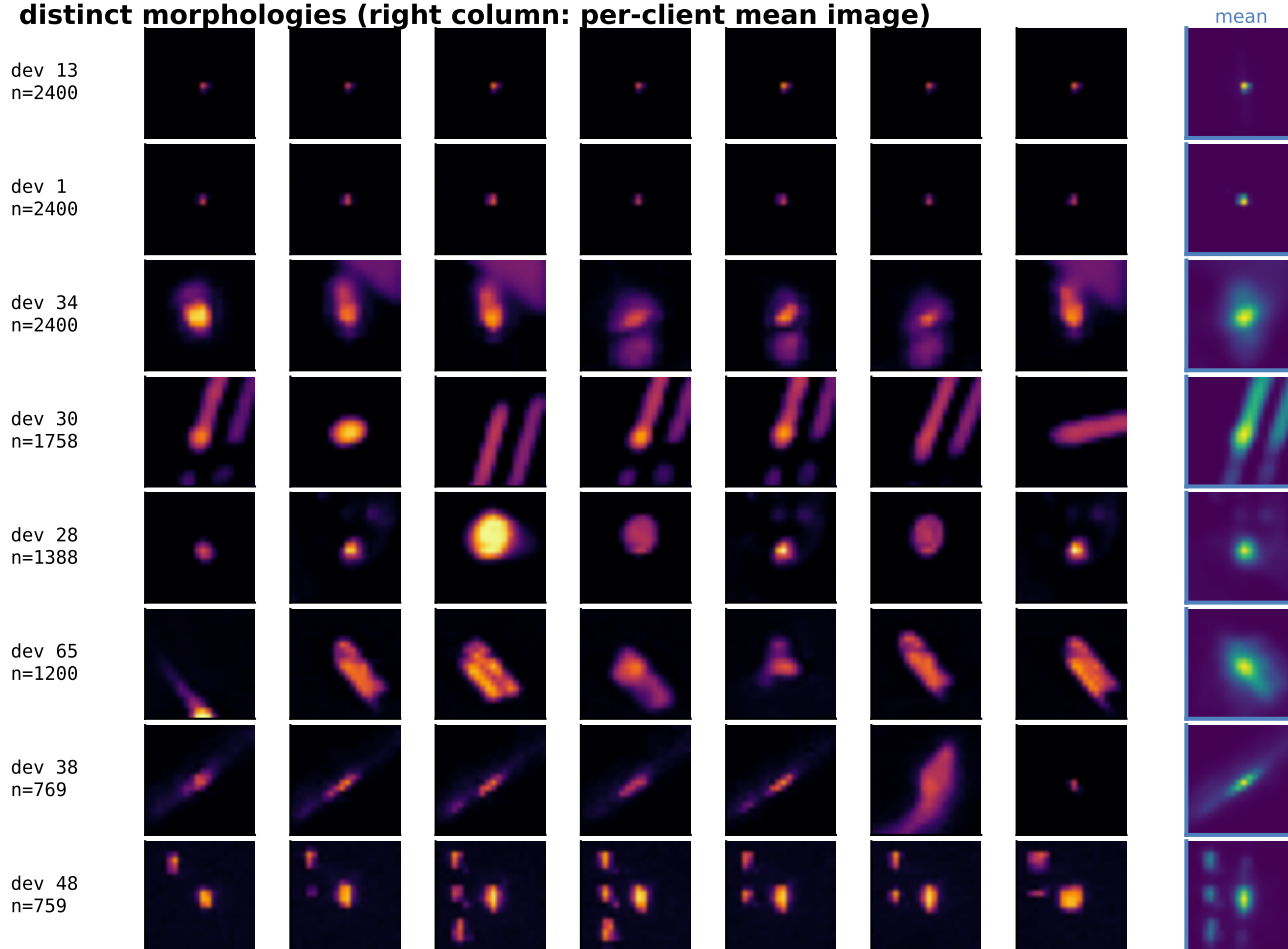
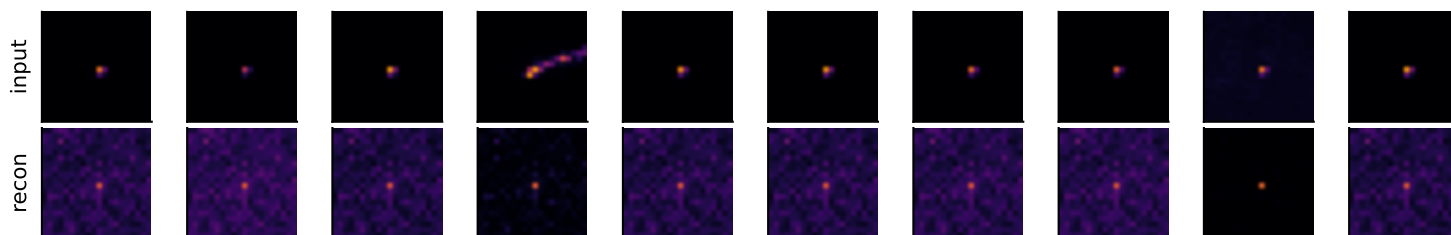


Fig. 11 — Held-out test images (top) vs autoencoder reconstruction (bottom) after the same 6-round run timed on the Pi 4 and Jetson (replicated locally, identical seeds/hyperparameters)



Inference headroom, gateway duty cycle, and the idle-power story

Fig. 5 — Inference latency (49-param MLP)
all boards $\sim 10^6$ - 10^7 x the detector rate

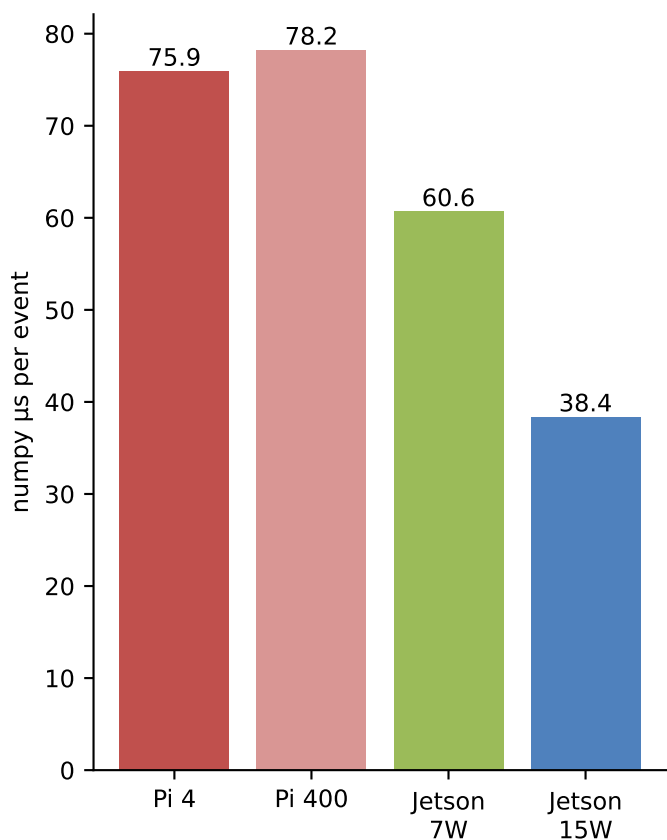


Fig. 6 — Gateway policy CPU cost on Pi 4
hybrid policy: 9,577x headroom, 0.04% duty
at the real 1.38 Hz event rate

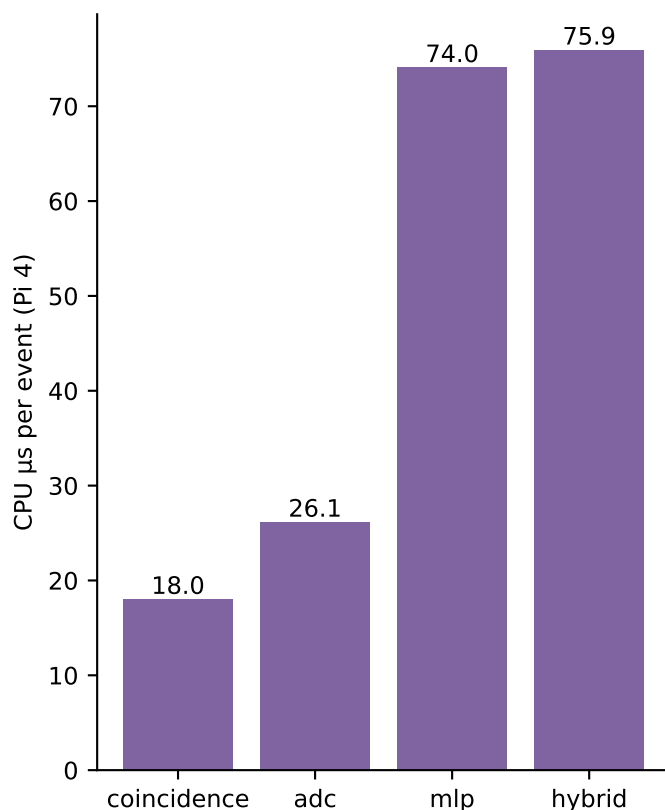


Fig. 7 — Jetson idle power vs software config
headless saves 0.04 W; 7W mode saves 0.16 W

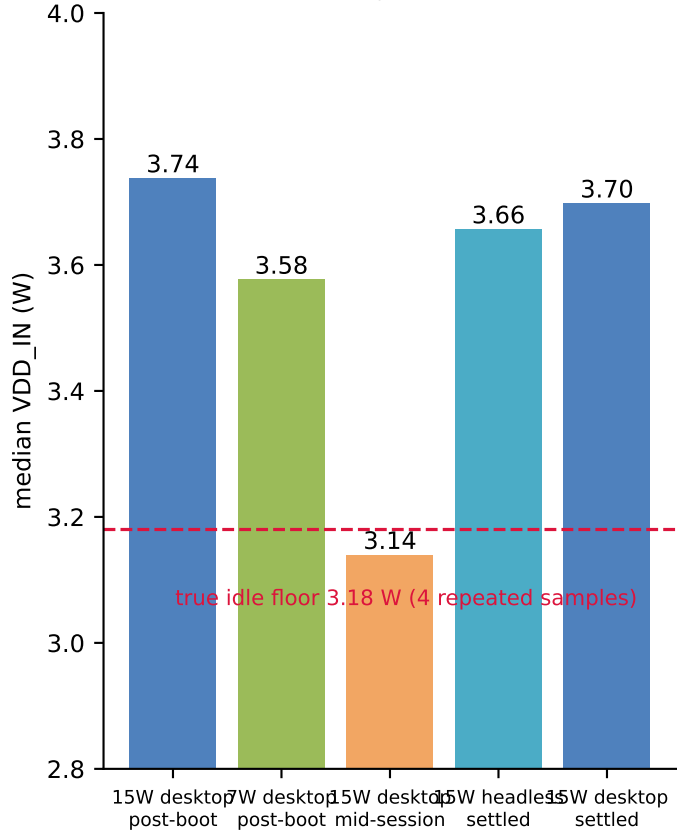
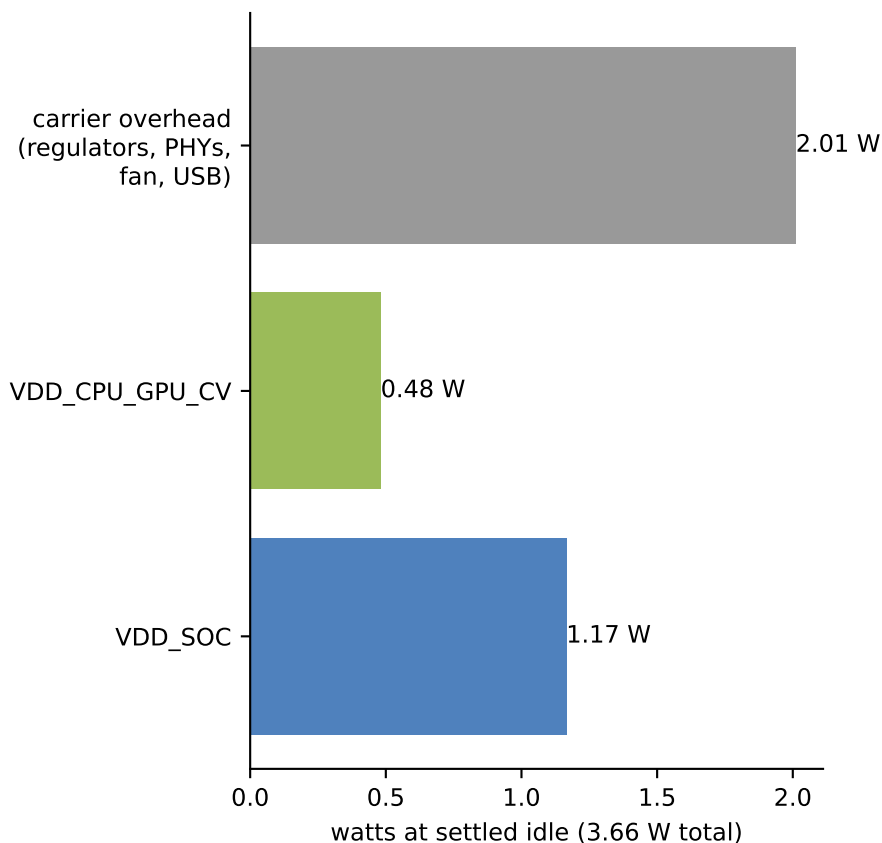


Fig. 8 — Where Jetson idle power goes
most is not software-addressable



Idle-power experiment detail and deployment decisions

Table 3 — Jetson Orin Nano idle-power measurements (INA3221 VDD_IN median, 5 Hz)

Condition	median W	window
15W desktop idle (2026-07-17, post-boot)	3.74	25 s
7W desktop idle (2026-07-17, post-boot)	3.58	25 s
15W desktop idle (2026-07-18, mid-session)	3.14	25 s
15W headless idle (settled, 2026-07-18)	3.66	60 s
15W desktop idle (settled, 2026-07-18)	3.70	60 s

Findings (revised 2026-07-18)

- True deep-idle floor: 3.18 W, rock-stable across four 30 s samples. The 3.66-3.74 W 'settled' readings were an elevated post-activity state (fan hysteresis suspected); idle has two states ~0.5 W apart.
- Desktop vs headless: null result (3.698 vs 3.657 W, ~1%) – GPU rail-gates either way.
- 7W nvpmode saves only ~0.16 W idle while giving up 37% of (unused) throughput.
- Conclusion: software cannot meaningfully reduce the dev-kit idle floor. Low-power deployment levers are hardware: custom carrier, duty-cycling/deep sleep, or moving the 49-parameter classifier to a microcontroller where idle is milliwatts.

Decision criteria (from EVENT_DRIVEN_HARDWARE.md)

- Prefer the Pi 4 as the always-on gateway: lower board power, and compute is irrelevant at a 0.04% duty fraction.
- Prefer the Jetson only when a GPU-sized workload (image training, realistic larger GNN) offsets its higher idle draw – the 26,584-param FL autoencoder does not qualify.
- Communication caveat: the 6-round FedAvg experiment transfers 1.95x the raw grayscale bytes, so compute speed alone cannot make the FL communication case.
- Open measurement: Pi 4 board watts need an inline USB-C meter (options and procedure documented in EVENT_DRIVEN_HARDWARE.md).

Caveats stated honestly

- Gateway selection fractions come from a synthetic stream – hardware cost numbers only.
- JetPack-7 PyPI torch wheel lacks CC 8.7 kernels; CUDA runs via PTX JIT (one-time cost, no steady-state penalty observed).
- 'Event-driven' means kernel-blocking software wakeup, not neuromorphic execution.